

Київський національний університет
імені Тараса Шевченка

Звіт

до лабораторної роботи №1
на тему:

**«Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 2,
варіант 4-6»**

Студента четвертого курсу
групи МІ-41
факультету комп'ютерних наук
та кібернетики
Петрова Богдана Вікторовича

Київ-2026

Зміст

1	Постановка задачі	1
2	Архітектура реалізації	1
3	Сервер	2
4	Десктоп-клієнт	6
4.1	Вхід і адреса сервера	6
4.2	Таблиця файлів і атрибути	6
4.3	Сортування за назвою (операція варіанта)	8
4.4	Фільтр за типом (операція варіанта)	8
4.5	Показ/приховування стовпців	8
4.6	Перегляд вмісту: .cs як текст, .jpg як зображення (тип варіанта)	8
4.7	Завантаження: кнопка та drag-and-drop (бонус)	12
4.8	Скачування і перетягування назовні (бонус)	12
4.9	Видалення	12
4.10	Прив'язка папки	13
4.11	Синхронізація та автоматичне відстеження	13
4.12	Правила синхронізації	13
5	Unit-тестування	14
6	Інструкція із запуску	15
7	Пакування	17
8	Висновки	17

1 Постановка задачі

Завдання етапу 2 — реалізувати десктоп-клієнт (Electron) і серверну частину (NestJS) системи MiniDrive для варіанта 4-6, спроектованої на етапі 1 (звіт `stage1-uml.md`): реєстрацію та вхід, перегляд файлів власного простору з атрибутами, сортування за назвою, фільтр «лише .cpp» / «лише .png», перегляд вмісту .cs як тексту та .jpg як зображення, завантаження/скачування/видалення файлів, синхронізацію локальної папки з віддаленим простором, а також обидва бонуси — drag-and-drop при завантаженні та скачуванні. Постановку задачі, аналіз вимог і UML-специфікацію детально описано на етапі 1; цей звіт фіксує їх реалізацію, unit-тести та результати запуску.

2 Архітектура реалізації

Проект — монорепозіторій на Yarn 4 workspaces (packageManager: yarn@4.9.1, nodeLinker: node-modules, без PnP):

- `packages/shared` — спільна TypeScript-логіка без залежностей від Electron, DOM чи Node: типи (`FileDto`, `UserDto`, `AuthResponseDto` тощо), `ApiClient` (обгортка над REST), чисті функції варіанта `sortByName/filterByType`, `previewKindOf/createPreview`, `toggleColumn`, `DriveViewModel` (фасад стану екрана) та `computeSyncPlan` (правила синхронізації). Пакет збирається у dual ESM/CJS і використовується і десктоп-, і майбутнім веб-клієнтом.
- `packages/ui` — презентаційні компоненти на React + shadcn/Tailwind (`FileTable`, `SortControl`, `FilterControl`, `ColumnToggle`, `PreviewPanel`, `UploadDropzone`), незалежні від платформи: усе платформозалежне передається пропсами.
- `apps/api` — сервер NestJS: модулі `auth`, `users`, `files`, `storage`, `prisma`.
- `apps/desktop` — Electron-застосунок (`electron-vite`): процеси `main` (вікно, IPC, `SyncEngine`, `FolderWatcher`, `LocalFolderScanner`, `DragOutHandler`), `preload` (`PreloadBridge`, тобто `window.minidrive`) і `renderer` (React-екрани `LoginScreen/DriveScreen` на компонентах `packages/ui`).

Відповідність діаграмам етапу 1: компонентна структура (рис. 1 етапу 1, `docs/uml/img/15-component.png`) один в один відповідає поділу на `packages/shared`, `apps/api` і три процеси `apps/desktop`; діаграма класів сервера (рис. 6, `docs/uml/img/04-class-server.png`) — контролерам/сервісам NestJS нижче; діаграма класів клієнтів (рис. 7, `docs/uml/img/05-class-clients.png`) — файлам `packages/shared/src/*.ts` і компонентам `packages/ui`.

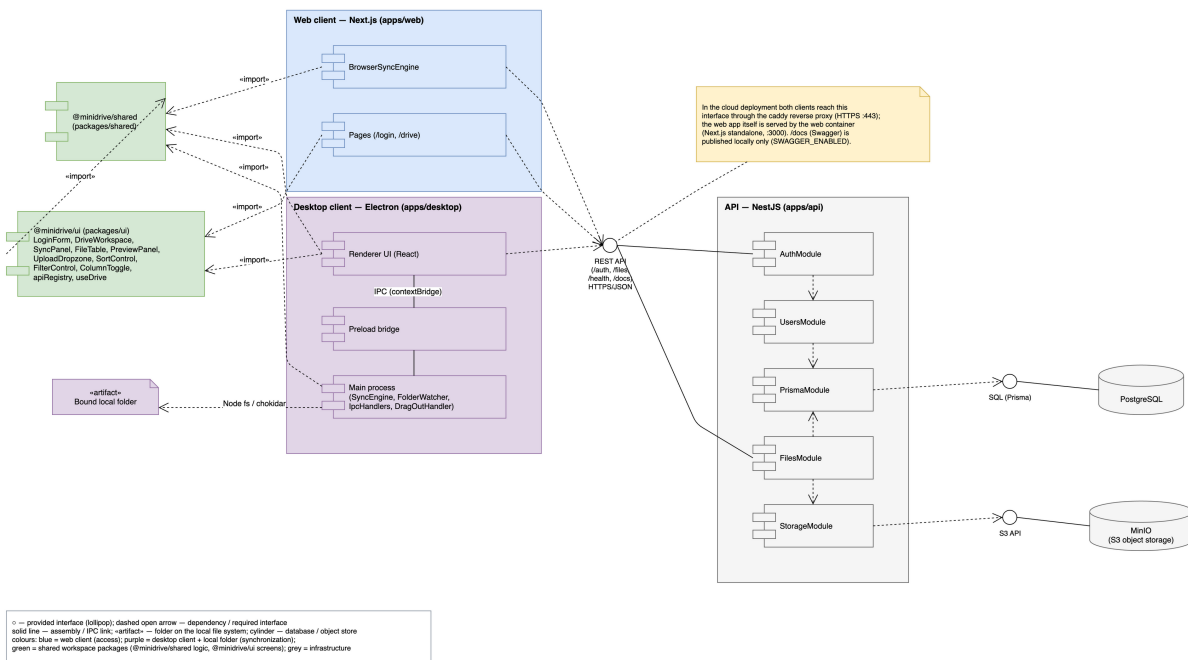


Рисунок 1. Діаграма компонентів (етап 1)

3 Сервер

REST API (NestJS, префікс без версії, порт 3000):

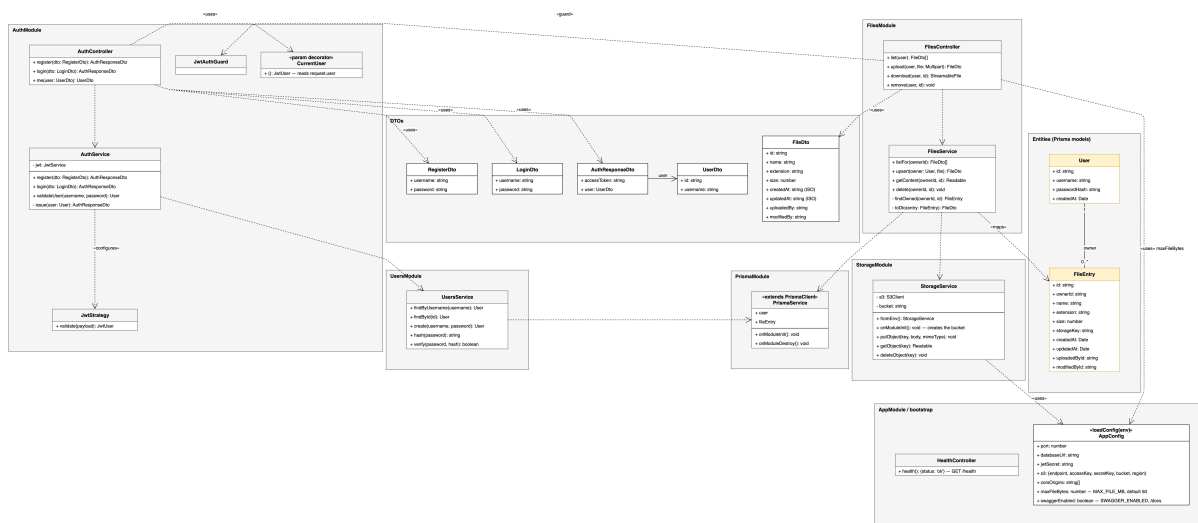


Рисунок 2. Діаграма класів сервера (етап 1)

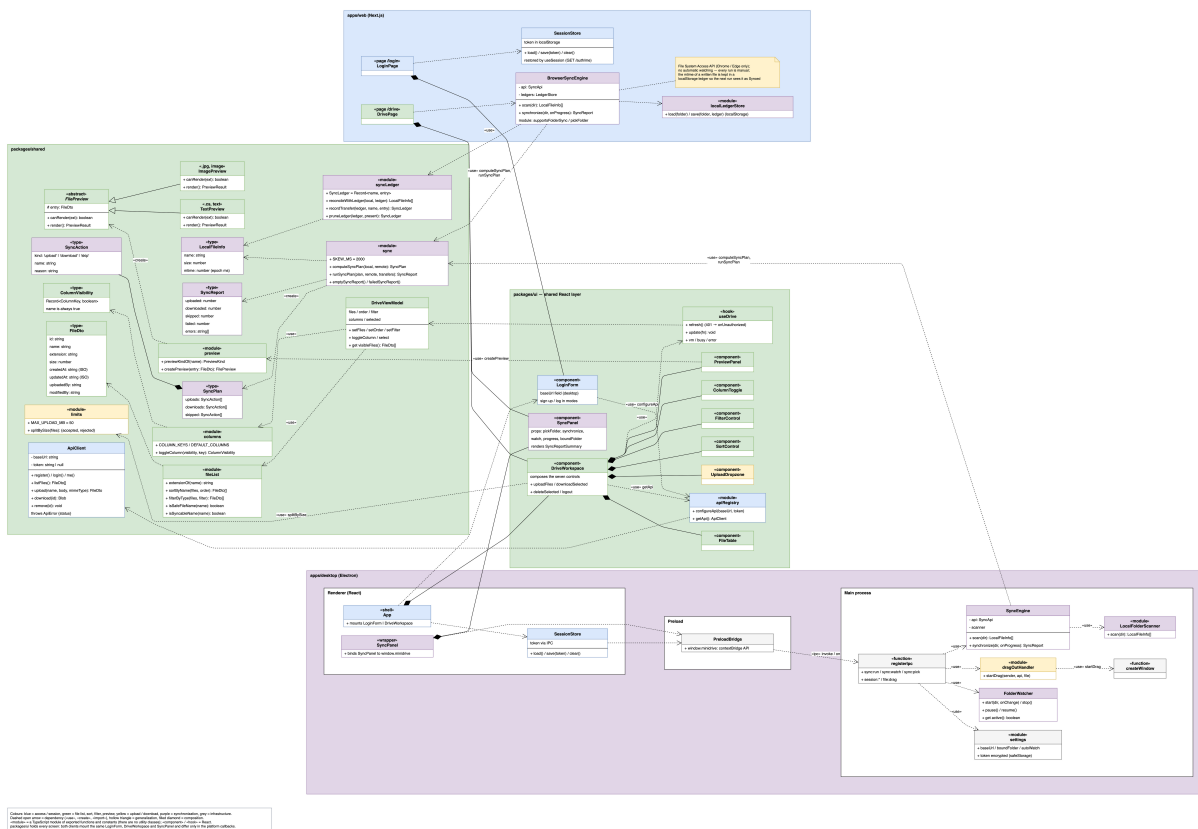


Рисунок 3. Діаграма класів спільного пакета і клієнтів (етап 1)

Метод і шлях	Призначення	Успіх	Помилки
POST /auth/register	реєстрація нового користувача	201, AuthResponseDto	409 — ім'я зайняте, 400 — валідація
POST /auth/login	вхід за логіном/паролем	200, AuthResponseDto	401 — невірний пароль
GET /auth/me	поточний користувач за токеном	200, UserDto	401
GET /files	список файлів власного простору	200, FileDto[]	401
POST /files	завантаження файлу (multipart, поле file)	201, FileDto	401, 413 — понад 50 МБ, 400
GET /files/:id/content	вміст файлу (стрім)	200, потік байтів	401, 404 — файл не в просторі викликача
DELETE /files/:id	видалення файлу	204	401, 404

Автентифікація — JWT HS256 з часом життя 24 год у заголовку `Authorization: Bearer <token>`; паролі хешуються bcrypt (10 раундів). Дані зберігаються в PostgreSQL через Prisma; модель `FileEntry` містить `ownerId`, `name`, `extension`, `size`, `storageKey`, `createdAt`, `updatedAt`, `uploadedById`, `modifiedById` і унікальний індекс (`ownerId`, `name`); модель `User` — `id`, `username` (унікальний), `passwordHash`.

Вміст файлів зберігається в MinIO (S3-сумісне сховище), бакет `minidrive`, ключ об'єкта — `users/{ownerId}/{fileEntry.id}`. `StorageService` інкапсулює `putObject/getObject/deleteObject` і за потреби створює бакет при старті (`onModuleInit`).

Правило перезапису: якщо завантажене ім'я вже існує у просторі того самого власника, `FilesService.upsert` не створює новий рядок, а лишає той самий `FileEntry.id` і `storageKey`, замінює байти в MinIO, ставить `updatedAt = now()` і `modifiedById = caller`. Це перевірено тестом `FilesService.upsert`: «overwrites an existing name: same id and key, new bytes, modifiedBy = caller» (`apps/api/src/files/files.service.spec.ts`).

Стек піднімається `docker compose (docker/compose.yml)`: сервіси `postgres`, `minio` та `api` (образ будується з `docker/api.Dockerfile`), усі з `healthcheck`. У робочому стані:

Специфікація REST-інтерфейсу автоматично публікується через `@nestjs/swagger` на `/docs`:

Наскрізна перевірка проти живого стеку — скрипт `tools/api-smoke.sh` (реєстрація/вхід відсутні, бо користувачі вже засіяні `db:seed`; сценарій: вхід → завантаження → список → скачування → видалення), який використано замість e2e-тесту `supertest` (узгоджене відхилення від §7 специфікації).

```
ai_avenger -- zsh — 154x48

NAME                IMAGE                COMMAND                SERVICE    CREATED      STATUS              PORTS
docker-api-1        docker-api           "docker-entrypoint.s..." api         21 minutes ago Up 21 minutes (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
docker-minio-1      minio/minio:latest  "/usr/bin/docker-ent..." minio       21 minutes ago Up 21 minutes (healthy) 0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
docker-postgres-1   postgres:16-alpine  "docker-entrypoint.s..." postgres   21 minutes ago Up 21 minutes (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
```

Рисунок 4. Термінал: docker compose ps, усі три сервіси healthy

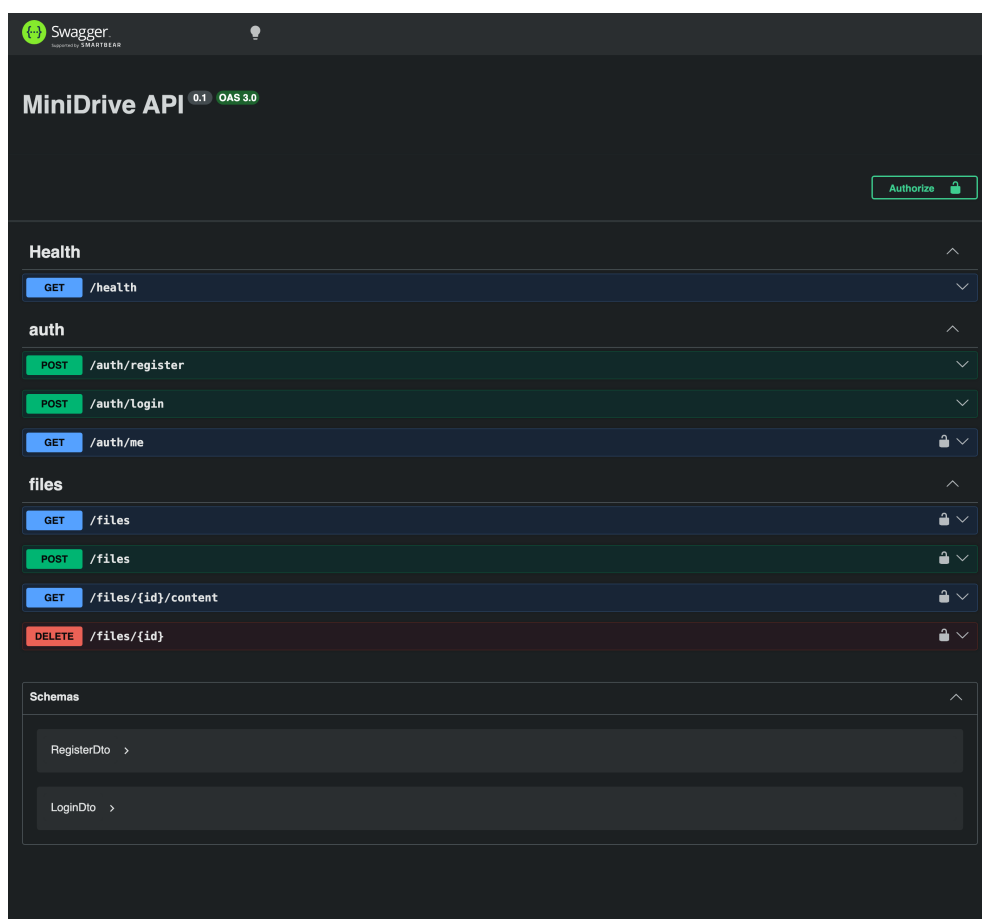


Рисунок 5. Swagger UI (<http://localhost:3000/docs>)

4 Десктоп-клієнт

Нижче — кожна функція десктоп-клієнта зі скриншотом. Знімки зроблено в чистому стані: вхід під bohdan, завантаження Program.cs, photo.jpg, main.cpp, logo.png, notes.txt, archive.zip, повторне завантаження Program.cs (щоб «Змінено» відрізнялося від «Створено»); після зйомки тестові файли видалено з сервера.

4.1 Вхід і адреса сервера

Екран входу (LoginScreen) має поле «Адреса сервера» (за замовчуванням `http://localhost:3000`, зберігається в `electron-store` через `SessionStore`), ім'я користувача й пароль; той самий екран перемикається в режим реєстрації. Змінюючи адресу сервера, клієнт можна спрямувати на інший (наприклад, опублікований в інтернеті) інстанс API без перезбирання застосунку.

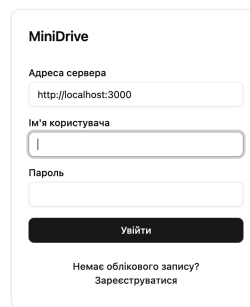


Рисунок 6. Екран входу з полем адреси сервера

4.2 Таблиця файлів і атрибути

`FileTable` показує «Назва», «Тип», «Розмір», «Створено», «Змінено», «Завантажив», «Редагував». Оскільки простори персональні і функція «поділитися» поза межами проєкту (*sharing out of scope*), у власному просторі «Завантажив» і «Редагував» завжди показують того самого користувача — це очікувана поведінка, а не помилка; стовпці все одно виведено обидва, щоб інтерфейс був готовий до можливого розширення. Значення розходяться, коли файл із тим самим іменем завантажується повторно: «Редагував» лишається як є (той самий викликач у персональному просторі), а «Змінено» оновлюється — новий файл Program.cs це демонструє (розділ «Синхронізація» нижче: «Змінено» 00:15:27 проти «Створено» 00:15:14).

MiniDrive
bohdan
Вийти

↑↓ Назва
Усі файли

☒ Тип
☒ Розмір
☒ Створено
☒ Змінено
☒ Завантажив
☒ Редагував
Оновити
Скачати
Видалити

/tmp/minidrive-watchtest
Змінити

Синхронізувати
☐ Автоматично відстежувати зміни

↑ Завантажити файли

Назва	Тип	Розмір	Створено	Змінено	Завантажив	Редагував
archive.zip	zip	299 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
logo.png	png	35.1 КБ	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
main.cpp	cpp	101 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
notes.txt	txt	181 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
photo.jpg	jpg	33.7 КБ	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
Program.cs	cs	192 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:27	bohdan	bohdan

Оберіть файл, щоб побачити вміст

Рисунок 7. Таблиця з усіма стовпцями, сортування за зростанням

4.3 Сортювання за назвою (операція варіанта)

SortControl перемикає порядок сортювання за клацанням на «Назва»; сортювання стабільне, реєстронезалежне, з `localeCompare(..., { numeric: true })` (`sortByName, packages/shared/src/fileList`).

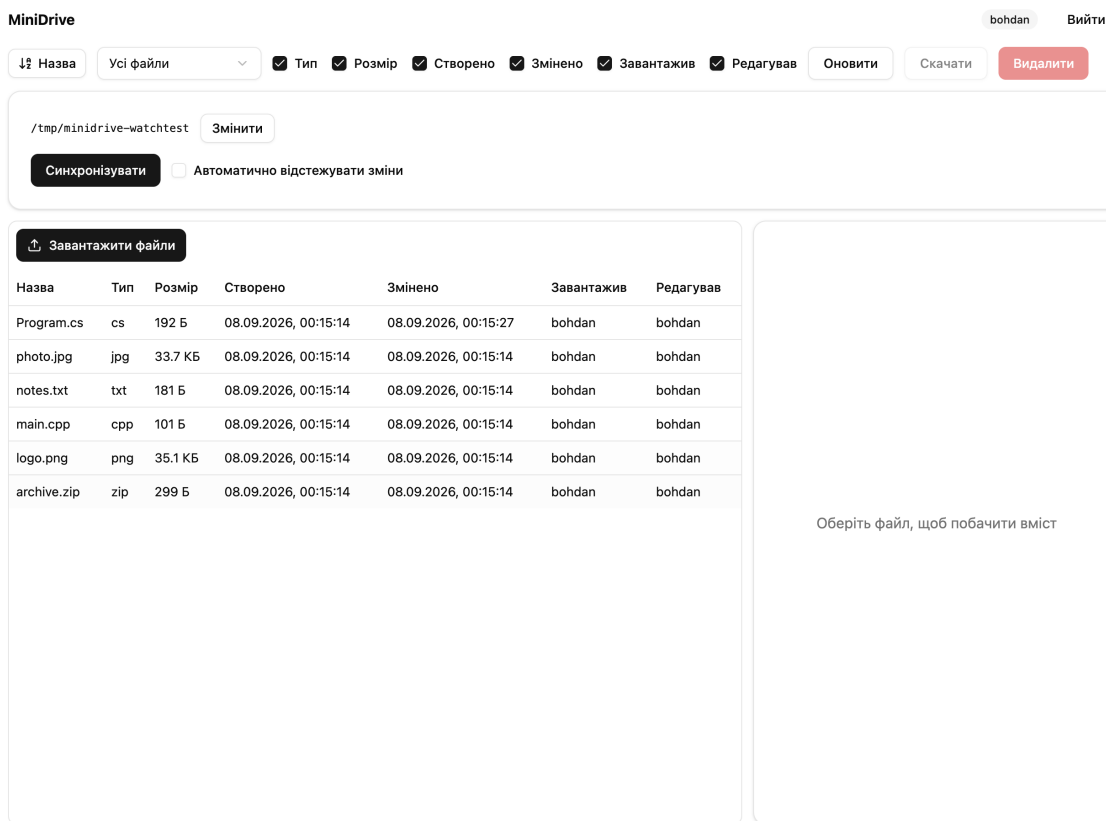


Рисунок 8. Сортювання за спаданням

4.4 Фільтр за типом (операція варіанта)

FilterControl — перемикач «Усі файли» / «Лише .cpp» / «Лише .png» (`filterByType`, реєстронезалежний за розширенням).

4.5 Показ/приховування стовпців

ColumnToggle — прапорці для всіх стовпців, крім «Назва», яку приховати неможливо (перевірено тестом `toggleColumn: «never hides the name column»`).

4.6 Перегляд вмісту: .cs як текст, .jpg як зображення (тип варіанта)

PreviewPanel викликає `createPreview/previewKindOf` із `packages/shared`: текстові розширення (зокрема .cs) рендеряться як текст, растрові зображення (зокрема .jpg) — як `<img>`.

MiniDrive bohdan Вийти

↑↓ Назва

Лише .png

☒ Тип
 ☒ Розмір
 ☒ Створено
 ☒ Змінено
 ☒ Завантажив
 ☒ Редагував

Оновити

Скачати

Видалити

Папку не обрано

Обрати папку

☐ Автоматично відстежувати зміни

Синхронізувати

Завантажити файли

Назва	Тип	Розмір	Створено	Змінено	Завантажив	Редагував
logo.png	png	35.1 КБ	08.09.2026, 09:55:40	08.09.2026, 09:55:40	bohdan	bohdan

Оберіть файл, щоб побачити вміст

Рисунок 9. Фільтр «Лише .png»: залишився logo.png

MiniDrive bohdan Вийти

↑↓ Назва

Усі файли

☐ Тип
 ☐ Розмір
 ☐ Створено
 ☒ Змінено
 ☐ Завантажив
 ☐ Редагував

Оновити

Скачати

Видалити

/tmp/minidrive-watchtest

Змінити

Синхронізувати

☐ Автоматично відстежувати зміни

Завантажити файли

Назва	Змінено
archive.zip	08.09.2026, 00:15:14
logo.png	08.09.2026, 00:15:14
main.cpp	08.09.2026, 00:15:14
notes.txt	08.09.2026, 00:15:14
photo.jpg	08.09.2026, 00:15:14
Program.cs	08.09.2026, 00:15:27

Program.cs

.cs

Створено

08.09.2026, 00:15:14

Змінено

08.09.2026, 00:15:27

Завантажив

bohdan

Редагував

bohdan

```
using System;

namespace MiniDrive
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, MiniDrive!
Version 2.");
        }
    }
}
```

Рисунок 10. Кілька стовпців приховано, «Назва» лишається видимою

MiniDrive
bohdan
Вийти

↑↓ Назва
Усі файли

☒ Тип
☒ Розмір
☒ Створено
☒ Змінено
☒ Завантажив
☒ Редагував

Оновити

Скачати

Видалити

/tmp/minidrive-watchtest
Змінити

Синхронізувати
☐ Автоматично відстежувати зміни

↑ Завантажити файли

Назва	Тип	Розмір	Створено	Змінено	Завантажив	Редагував
archive.zip	zip	299 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
logo.png	png	35.1 КБ	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
main.cpp	cpp	101 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
notes.txt	txt	181 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
photo.jpg	jpg	33.7 КБ	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
Program.cs	cs	192 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:27	bohdan	bohdan

Program.cs

.cs

Створено

08.09.2026, 00:15:14

Змінено

08.09.2026, 00:15:27

Завантажив

bohdan

Редагував

bohdan

```
using System;
namespace MiniDrive
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, MiniDrive!
Version 2.");
        }
    }
}
```

Рисунок 11. Перегляд .cs як тексту

MiniDrive
bohdan
Вийти

↑↓ Назва
Усі файли

☒ Тип
☒ Розмір
☒ Створено
☒ Змінено
☒ Завантажив
☒ Редагував

Оновити

Скачати

Видалити

/tmp/minidrive-watchtest
Змінити

Синхронізувати
☐ Автоматично відстежувати зміни

↑ Завантажити файли

Назва	Тип	Розмір	Створено	Змінено	Завантажив	Редагував
archive.zip	zip	299 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
logo.png	png	35.1 КБ	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
main.cpp	cpp	101 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
notes.txt	txt	181 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
photo.jpg	jpg	33.7 КБ	08.09.2026, 00:15:14	08.09.2026, 00:15:14	bohdan	bohdan
Program.cs	cs	192 Б	08.09.2026, 00:15:14	08.09.2026, 00:15:27	bohdan	bohdan

photo.jpg
.jpg

Створено

08.09.2026, 00:15:14

Змінено

08.09.2026, 00:15:14

Завантажив

bohdan

Редагував

bohdan

Рисунок 12. Перегляд .jpg як зображення

4.7 Завантаження: кнопка та drag-and-drop (бонус)

UploadDropzone поєднує кнопку «Завантажити файли» (прихований `<input type="file" multiple>`) і зону перетягування: `onDragOver` показує підказку «Відпустіть, щоб завантажити», `onDrop` передає файли в той самий обробник завантаження, що й кнопка.

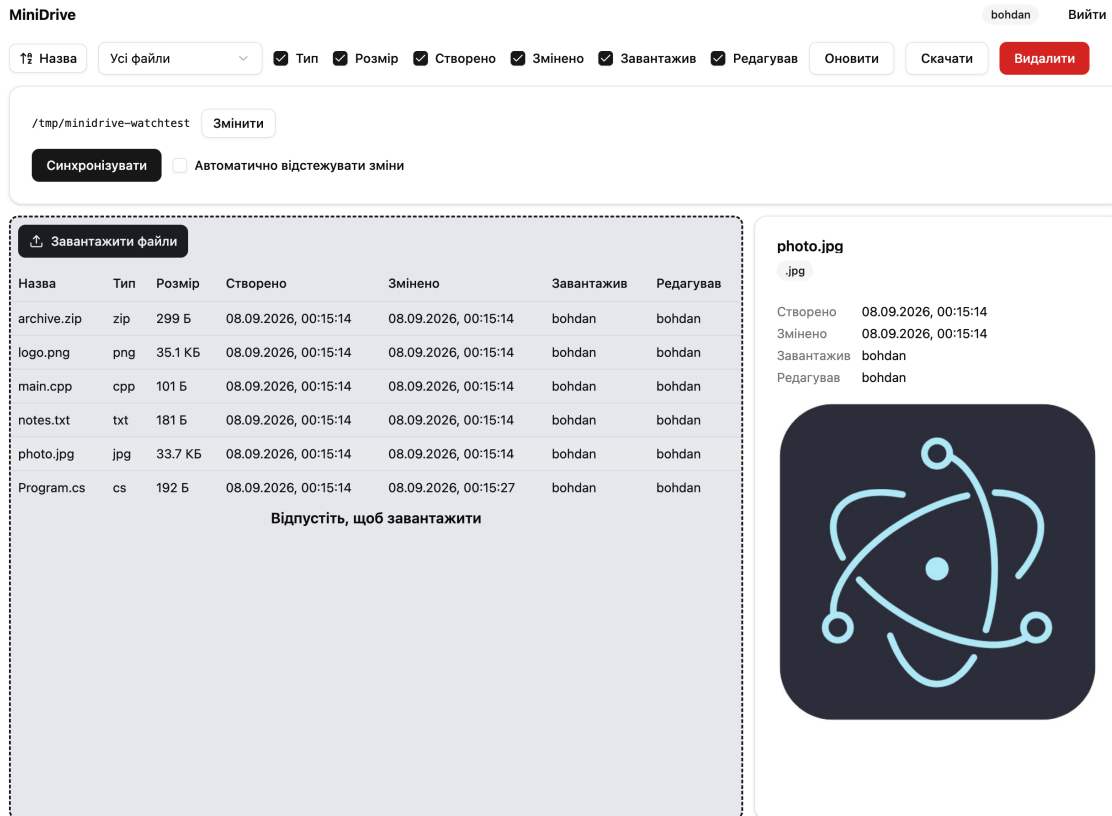


Рисунок 13. Підказка під час перетягування файлів у вікно

4.8 Скачування і перетягування назовні (бонус)

Кнопка «Скачати» зберігає вміст обраного файлу через нативний діалог збереження (`window.minidrive.file` головний процес, `dialog.showSaveDialog`). Перетягування рядка таблиці назовні вікна обробляє `DragOutHandler` у головному процесі (`window.minidrive.file.dragOut`) — Electron записує тимчасовий файл і починає системний drag, тож файл можна відпустити у Finder або іншому застосунку.

4.9 Видалення

Кнопка «Видалити» відкриває діалог підтвердження (`Dialog/DialogFooter` з `packages/ui`) і після підтвердження викликає `DELETE /files/:id`.

4.10 Прив'язка папки

Панель синхронізації (SyncPanel) дозволяє прив'язати локальну папку через нативний діалог вибору директорії (`window.minidrive.folder.pick`, `dialog.showOpenDialog` з `openDirectory`); шлях зберігається в налаштуваннях і показується поруч із кнопкою «Змінити».

4.11 Синхронізація та автоматичне відстеження

Кнопка «Синхронізувати» викликає `SyncEngine.synchronize(): LocalFolderScanner.scan()` читає локальну папку, `ApiClient.listFiles()` — віддалений список, `computeSyncPlan` будує план дій, які виконуються послідовно; результат — `SyncReport` (`uploaded`, `downloaded`, `skipped`, `failed`, `errors`), показаний у панелі. Прапорець «Автоматично відстежувати зміни» вмикає `FolderWatcher.startWatching()` (`chokidar`, дебаунс 2 с): кожна зміна у прив'язаній папці запускає нову синхронізацію без участі користувача; `stopWatching()` вимикає стеження.

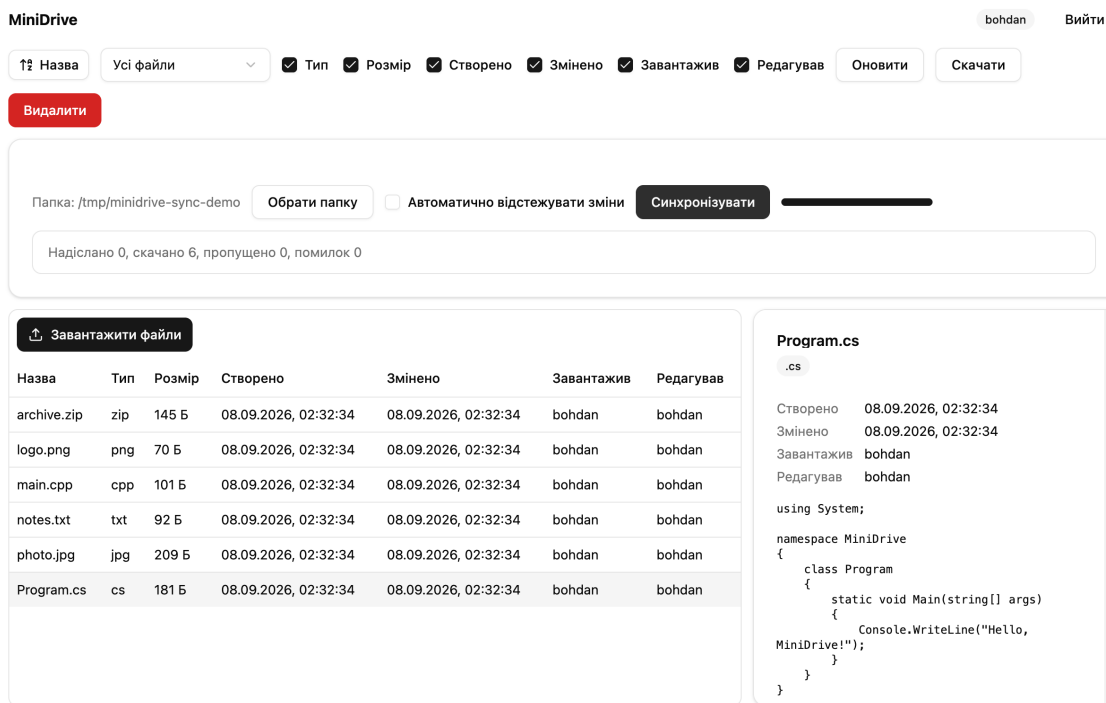


Рисунок 14. Панель синхронізації після натискання «Синхронізувати»: рядок звіту «Надіслано 0, скачано 6, пропущено 0, помилок 0» — локальних файлів для відправки не було, усі 6 файлів скачано на щойно прив'язану порожню локальну папку

4.12 Правила синхронізації

Реалізовано в `computeSyncPlan` (`packages/shared/src/sync.ts`) і покрито тестами `sync.test.ts` та `syncEngine.test.ts`: файл лише локально → надсилається; файл лише віддалено → скачується; за наявності з обох боків з однаковим розміром і $|mtime - updatedAt| \leq 2000$ мс — пропускається; інакше перемагає новіша версія (за `mtime/updatedAt`); видалення ніколи не поширюються в інший

бік (видалення локально відновлює файл з сервера і навпаки); після скачування локальний mtime виставляється рівним updatedAt сервера, щоб наступна синхронізація не вважала файл зміненим.

5 Unit-тестування

Файли тестів подано як «робочий простір / файл»: shared — packages/shared/src, api — apps/api/src (з підкаталогами модулів), desktop — apps/desktop/src/main/sync.

Файл тестів	Що перевіряє	К-сть
shared / fileList.test.ts	sortByName (зростання/спадання, стабільність, реєстронезалежність — операція варіанта), filterByType (усі файли / лише .cpp / лише .png — операція варіанта , окремо копія масиву для 'all'), extensionOf (звичайне розширення й порожнє для імені, що закінчується голою крапкою), isSafeFileName, isSyncableName	12
shared / preview.test.ts	previewKindOf для типів варіанта .cs→текст, .jpg→зображення, узагальнено для інших текстових/растрових, 'none' для інших; createPreview; FilePreview.canRender	7
shared / columns.test.ts	toggleColumn, незнімний стовпець «Назва»	2
shared / driveViewModel.test.ts	DriveViewModel: застосування сортування й фільтра варіанта до visibleFiles, неможливість приховати «Назва»	2
shared / apiClient.test.ts	ApiClient: bearer-токен, ApiError зі статусом, multipart з полем file, обрізання кінцевого слеша базової адреси, скачування як blob, видалення файлу, реєстрація, undefined на відповідь 204, об'єднання масиву повідомлень помилки	9
shared / sync.test.ts	computeSyncPlan: лише локально/лише віддалено, пропуск у межах похибки годинника, перемога новішої версії, різниця розміру як зміна, скачування замість помилки при непридатній для розбору мітці часу сервера; emptySyncReport, failedSyncReport	9
api / auth.service.spec.ts	реєстрація, 409 на зайняте ім'я, 401 на невірний пароль, вхід	4
api / jwt.strategy.spec.ts	JwtStrategy.validate: повертає поточного користувача, відхиляє токен видаленого користувача	2
api / config.spec.ts	loadConfig: типовий порт, коли PORT не число	1

Файл тестів	Що перевіряє	К-сть
api / files.controller.spec.ts	FilesController.upload: 400 на завантаження без файлу	1
api / files.service.spec.ts	FilesService.upsert (створення, перезапис з тим самим id/key), валідація імені, чужий простір (404), відкат рядка при помилці сховища, порядок видалення	8
api / storage.service.spec.ts	StorageService.putObject/getObject/deleteObject, 3 створення бакета при відсутності	3
api / users.service.spec.ts	bcrypt-хеш пароля, пошук за іменем	2
desktop / syncEngine.test.ts	SyncEngine.scan/synchronize: завантаження та скачування зі встановленням mtime, пропуск ідентичних, підрахунок помилок без переривання, захист від виходу за межі каталогу (шлях і зворотний слеш), пропуск скачування файлу з крапкою на початку імені, звіт про прогрес (onProgress)	7

Разом: **shared — 41, api — 21, desktop — 7**, усього 69 тестів, усі проходять (yarn test з кореня):

6 Інструкція із запуску

```
corepack enable
yarn install
cp docker/.env.example docker/.env
yarn stack:up
yarn workspace @minidrive/api db:seed
yarn workspace @minidrive/desktop dev
```

yarn stack:up піднімає postgres, minio та api в Docker (docker compose -f docker/compose.yml up -d --build --wait) і чекає на healthcheck усіх трьох сервісів; yarn workspace @minidrive/api db:seed створює тестових користувачів (bohdan, olena, taras, пароль secret123). yarn workspace @minidrive/desktop dev запускає Electron-клієнт у режимі розробки (electron-vite); yarn workspace @minidrive/desktop build:mac (або build:win/build:linux) збирає розповсюджуваний застосунок.

Змінні середовища сервера (docker/.env, копія з docker/.env.example, у git не потрапляє): POSTGRES_USER/POSTGRES_PASSWORD/POSTGRES_DB, MINIO_ROOT_USER/MINIO_ROOT_PASSWORD, JWT_SECRET, API_PORT, CORS_ORIGINS.

Щоб підключити десктоп-клієнт до віддаленого (не локального) сервера, достатньо на екрані входу вказати в полі «Адреса сервера» URL опублікованого API (наприклад, <https://minidrive.example.com>)

```
ai_avenger --zsh -- 154x48

Test Suites: 7 passed, 7 total
Tests:       21 passed, 21 total
Snapshots:   0 total
Time:        0.565 s, estimated 1 s
Ran all test suites.

RUN v5.0.0 apps/desktop

Test Files  1 passed (1)
Tests       7 passed (7)
Start at    00:13:45
Duration    96ms (transform 65%, import 16%, tests 16%, worker 3%)

RUN v3.2.7 packages/shared

✓ src/sync.test.ts (9 tests) 3ms
✓ src/columns.test.ts (2 tests) 1ms
✓ src/preview.test.ts (7 tests) 3ms
✓ src/fileList.test.ts (12 tests) 9ms
✓ src/apiClient.test.ts (9 tests) 6ms
✓ src/driveViewModel.test.ts (2 tests) 7ms

Test Files  6 passed (6)
Tests       41 passed (41)
Start at    00:13:46
Duration    214ms (transform 83ms, setup 0ms, collect 146ms, tests 29ms, environment 0ms, prepare 268ms)

Done in 2s 619ms
```

Рисунок 15. Термінал з результатом yarn test: 7 test suites api / 1 test file desktop / 6 test files shared, усі passed

замість `http://localhost:3000` — значення зберігається в `electron-store` й використовується для всіх наступних запитів `ApiClient`, перезбирання застосунку не потрібне.

7 Пакування

`yarn workspace @minidrive/desktop build:mac (electron-builder)` успішно зібрав застосунок під ідентифікатором `appId: ua.knu.minidrive` і назвою `productName: MiniDrive`; підписання коду вимкнено `identity: null` у розділі `mac apps/desktop/electron-builder.yml` (сертифікатів розробника в системі немає — очікувано для навчального середовища), а шаблон імені артефактів (`nsis/mac/dmg: artifactName: ${productName}-${version}-${arch}.${ext}`) явно використовує `productName` замість повного, зі скоупом, імені пакета. Перша спроба збірки `.dmg` завершувалася помилкою, бо типовий шаблон `electron-builder` підставляв саме ім'я пакета `@minidrive/desktop` (зі слешем зі скоупу) у шлях файлу; після явного завдання `artifactName` збірка проходить повністю. Результат: `apps/desktop/dist/MiniDrive-1.0.0-arm64.dmg` і `apps/desktop/dist/MiniDrive-1.0.0-arm64.zip`.

8 Висновки

На етапі 2 реалізовано REST-сервер (`NestJS`, `PostgreSQL`, `MinIO`) і десктоп-клієнт (`Electron`) системи `MiniDrive` за специфікацією етапу 1: усі функціональні вимоги типу 2, обидві операції варіанта (сортування за назвою, фільтр `.cpp/.png`), обидва типи перегляду варіанта (`.cs`, `.jpg`) і обидва бонуси (`drag-and-drop` при завантаженні та скачуванні) реалізовано й перевірено вручну через 12 знімків екрана. Поведінку ключових функцій — сортування, фільтрації, перегляду, атрибутів перезапису та правил синхронізації — покрито 69 unit-тестами (`shared` 41, `api` 21, `desktop` 7), усі проходять. Наскрізну працездатність стеку підтверджує `tools/api-smoke.sh` проти запущеного `docker compose`. Пакування в розповсюджуваний `MiniDrive-1.0.0-arm64.dmg/.zip` для `macOS` пройшло повністю успішно. Наступний етап (3) — веб-орієнтована версія клієнта на тому самому пакеті `packages/shared`.